

Drone-Based Delivery Optimization: A Combinatorial Optimization Approach Using Exact and Metaheuristic Methods

AHMED FOUATIH Hamza Faiz, Youcef Belaib, Dhiaa Eddine Zeroual, Ishak Dib

Department of Intelligent Systems Engineering
National School of Artificial Intelligence (ENSIA)
Algiers, Algeria

{hamza.faiz.ahmedfouatih, youcef.belaib, dhiaaeddine.zeroual, ishak.dib}@ensia.edu.dz

Abstract—This paper addresses the Drone-Based Delivery Optimization Problem (DDOP), a practical variant of the Capacitated Vehicle Routing Problem (CVRP) arising in last-mile logistics. Given a fleet of drones operating from a central depot, the objective is to design delivery routes minimizing total energy consumption while satisfying payload capacity, battery endurance, and no-fly zone constraints. We propose two distinct mathematical formulations: an arc-flow Mixed Integer Linear Programming (MILP) model based on the classical CVRP, and a set-partitioning flow-based model offering a different level of abstraction. We establish the NP-Hardness of the problem via reduction from Bin Packing. Three solution methods are implemented and compared: a Branch-and-Bound (B&B) exact solver with an admissible assignment-relaxation lower bound, a Genetic Algorithm (GA) using a Giant Tour encoding with Order Crossover, and a Simulated Annealing (SA) algorithm with four problem-specific neighborhood operators. Experiments are conducted on ten benchmark instances ranging from $n = 5$ to $n = 75$ customers. Results show that both metaheuristics produce high-quality feasible solutions; GA outperforms SA on larger instances while SA is competitive on medium-sized ones.

Index Terms—Drone delivery, Vehicle routing problem, Branch and bound, Genetic algorithm, Simulated annealing, Combinatorial optimization, Metaheuristics.

I. INTRODUCTION

The rapid growth of e-commerce and advances in autonomous aerial systems have made drone-based last-mile delivery a commercially viable and actively researched topic. Companies such as Amazon, DHL, and Wing have already deployed experimental drone delivery programs, and regulatory frameworks are progressively being established worldwide to support larger-scale operations.

From an operations research perspective, planning optimal drone delivery routes is a challenging combinatorial problem. Unlike traditional vehicle routing, drones face additional constraints: battery endurance strictly limits the range per flight, payload capacity bounds the weight carried per trip, and regulatory no-fly zones restrict certain airspace corridors. These constraints, combined with the need to serve all customers exactly once from a single depot, create an optimization problem that is both practically relevant and theoretically rich.

This work studies the Drone-Based Delivery Optimization Problem (DDOP) as assigned in the ENSIA third-year Combi-

natorial Optimization and Metaheuristics course project. The contributions of this paper are as follows:

- Two distinct mathematical formulations at different abstraction levels.
- A formal complexity analysis establishing NP-Hardness.
- A Branch-and-Bound exact solver with a tight admissible lower bound.
- A Genetic Algorithm with a Giant Tour representation and problem-specific operators.
- A Simulated Annealing algorithm with four neighborhood operators and an adaptive reheating mechanism.
- An experimental study on ten instances with full comparative analysis.

The remainder of this paper is structured as follows. Section II formally defines the problem. Section III presents both mathematical formulations. Section IV analyzes problem complexity. Section V describes the Branch-and-Bound method. Section VI details the Genetic Algorithm. Section VII details the Simulated Annealing. Section VIII presents the experimental study. Section IX concludes.

II. PROBLEM DEFINITION

A. Setting

A logistics company operates a homogeneous fleet of K drones. All drones depart from and must return to a single central *depot* (node 0). There are n customers (nodes $1, \dots, n$), each requiring delivery of a package of known weight $q_i > 0$.

The operational environment is represented as a complete directed graph $G = (V, A)$ where $V = \{0, 1, \dots, n\}$ and A is the set of allowed directed arcs. Each arc $(i, j) \in A$ has an associated *energy cost* $d_{ij} \geq 0$, modeled as the Euclidean distance between nodes i and j . Certain arcs are *forbidden* due to regulatory no-fly zones; these are removed from A (equivalently, assigned infinite cost).

B. Constraints

A feasible solution must satisfy the following constraints:

- 1) **Coverage:** Every customer $i \in \{1, \dots, n\}$ is visited by exactly one drone on exactly one route.

- 2) **Depot return:** Every route starts and ends at the depot.
- 3) **Payload capacity:** The total weight of packages carried on any single route must not exceed Q kg.
- 4) **Battery capacity:** The total energy consumed (total distance flown) on any single route must not exceed B .
- 5) **No-fly zones:** Only arcs in A may be traversed.

C. Objective

The goal is to minimize the total energy consumption across all drone routes:

$$\min \sum_{k=1}^K \text{cost}(r_k) \quad (1)$$

where $\text{cost}(r_k)$ denotes the total Euclidean distance of route r_k .

D. Assumptions

Table I summarizes the modeling assumptions used throughout this work.

TABLE I: Modeling Assumptions

Parameter	Value / Model
Energy model	Proportional to Euclidean distance
Depot location	(50, 50) in a $[0, 100]^2$ grid
Customer positions	Uniform random in $[0, 100]^2$
Demand q_i	Uniform in $[1, 10]$ kg
Payload Q	$\approx 3 \times \bar{q}$ (scales with instance)
Battery B	$\approx 2.5 \times \max_i d(0, i)$
No-fly zones	Customer-to-customer edges only
Fleet size K	Unbounded (opened as needed)

III. MATHEMATICAL FORMULATIONS

We present two fundamentally different formulations of the DDOP. They are *not* equivalent reformulations of the same model — they differ in variable space, constraint structure, and the algorithmic paradigms they naturally suggest.

A. Formulation 1: Arc-Flow MILP (CVRP Model)

This formulation is a direct adaptation of the classical Capacitated Vehicle Routing Problem (CVRP) [1]. Decision variables are defined at the *arc level*, encoding which drone traverses each edge.

1) Decision Variables:

$$\begin{aligned} x_{ijk} &\in \{0, 1\} && \text{1 if drone } k \text{ travels from } i \text{ to } j \\ u_{ik} &\in \mathbb{R}_{\geq 0} && \text{cumulative load of drone } k \text{ at node } i \end{aligned}$$

2) Objective:

$$\min \sum_{k \in K} \sum_{(i,j) \in A} d_{ij} \cdot x_{ijk} \quad (2)$$

3) Constraints: (C1) Each customer visited exactly once:

$$\sum_{k \in K} \sum_{j \in V, j \neq i} x_{ijk} = 1 \quad \forall i \in \{1, \dots, n\} \quad (3)$$

(C2) Flow conservation:

$$\sum_{j \neq i} x_{ijk} = \sum_{j \neq i} x_{jik} \quad \forall i \in V, \forall k \in K \quad (4)$$

(C3) Each drone departs from the depot at most once:

$$\sum_{j=1}^n x_{0jk} \leq 1 \quad \forall k \in K \quad (5)$$

(C4) Payload capacity:

$$\sum_{i=1}^n q_i \sum_{j \neq i} x_{ijk} \leq Q \quad \forall k \in K \quad (6)$$

(C5) Battery capacity:

$$\sum_{(i,j) \in A} d_{ij} \cdot x_{ijk} \leq B \quad \forall k \in K \quad (7)$$

(C6) Subtour elimination (Miller–Tucker–Zemlin):

$$u_{ik} - u_{jk} + Q \cdot x_{ijk} \leq Q - q_j \quad \forall i \neq j \in \{1..n\}, \forall k \quad (8)$$

$$q_i \leq u_{ik} \leq Q \quad \forall i \in \{1, \dots, n\}, \forall k \quad (9)$$

(C7) No-fly zone enforcement:

$$x_{ijk} = 0 \quad \forall (i, j) \notin A, \forall k \in K \quad (10)$$

(C8) Binary and non-negativity:

$$x_{ijk} \in \{0, 1\}, \quad u_{ik} \geq 0 \quad (11)$$

This model has $O(n^2K)$ binary variables and $O(n^2K)$ constraints. The MTZ constraints (8)–(9) eliminate subtours by enforcing a consistent load ordering: if drone k visits i before j , then $u_{jk} \geq u_{ik} + q_j$.

B. Formulation 2: Set-Partitioning / Flow Model

The second formulation operates at the *route level* rather than the arc level. It is inspired by the Dantzig–Wolfe decomposition approach to VRP, which forms the basis of column generation methods [5].

1) *Preprocessing:* Let $\mathcal{R}$ denote the set of all *feasible routes*. A route $r = (c_1, c_2, \dots, c_p)$ is feasible if:

- $\sum_{\ell=1}^p q_{c_\ell} \leq Q$ (payload constraint)
- $d(0, c_1) + \sum_{\ell=1}^{p-1} d(c_\ell, c_{\ell+1}) + d(c_p, 0) \leq B$ (battery constraint)
- All arcs $(c_\ell, c_{\ell+1}) \in A$ (no-fly constraint)

For each route $r \in \mathcal{R}$, define:

- c_r : total energy cost of route r
- $a_{ir} \in \{0, 1\}$: 1 if customer i is visited in route r

2) Decision Variable:

$$y_r \in \{0, 1\} \quad \text{1 if route } r \text{ is selected}$$

3) Objective:

$$\min \sum_{r \in \mathcal{R}} c_r \cdot y_r \quad (12)$$

4) *Constraints:* **(D1) Each customer covered exactly once:**

$$\sum_{r \in \mathcal{R}} a_{ir} \cdot y_r = 1 \quad \forall i \in \{1, \dots, n\} \quad (13)$$

(D2) Fleet size:

$$\sum_{r \in \mathcal{R}} y_r \leq K \quad (14)$$

(D3) Binary:

$$y_r \in \{0, 1\} \quad \forall r \in \mathcal{R} \quad (15)$$

5) *Comparison of the Two Formulations:* Table II contrasts the two models across key dimensions.

TABLE II: Comparison of Formulation 1 vs. Formulation 2

Aspect	F1 (Arc-Flow)	F2 (Set-Partition)
Variables	$O(n^2 K)$ binary arc vars	$O(\mathcal{R})$ route vars
Subtours	Explicit MTZ constraints	Implicit in route construction
No-fly zones	Arc removal from A	Route feasibility filter
LP relaxation	LP on fractional arcs	Column generation
Abstraction	Edge level	Path/route level
Scalability	Better for small n	Better LP bounds
Algorithm	B&B with cutting planes	Column generation + B&B

The two formulations represent *different abstraction levels*: Formulation 1 reasons about individual arc traversals and requires explicit subtour elimination, while Formulation 2 reasons about complete routes and avoids subtour constraints entirely at the cost of an exponentially large variable space.

IV. COMPLEXITY ANALYSIS

A. NP-Membership

Given a proposed solution (a set of routes), feasibility can be verified in polynomial time: check that each customer appears exactly once ($O(nK)$), that each route satisfies payload and battery constraints ($O(n)$ per route), and that no forbidden arc is used ($O(n \cdot |\text{no-fly}|)$). Thus DDOP $\in$ NP.

B. NP-Hardness

The Drone-Based Delivery Optimization Problem is NP-Hard.

We reduce from the NP-Complete problem BIN-PACKING [2].

Instance of BIN-PACKING: n items with sizes $s_1, s_2, \dots, s_n \in (0, 1]$ and bin capacity C . Can all items be packed into at most m bins?

Reduction to DDOP: Construct a DDOP instance as follows:

- Place all n customers at the same location as the depot (zero distances, so $d_{ij} = 0$ for all i, j).
- Set $q_i = s_i$ (demand equals item size).
- Set payload capacity $Q = C$ and battery capacity $B = +\infty$.
- Set fleet size $K = m$.

Equivalence: A feasible drone assignment exists with at most m drones iff the items fit into m bins of capacity C . This reduction is polynomial (linear time).

Since BIN-PACKING is NP-Complete and DDOP $\in$ NP, DDOP is NP-Complete (and in particular NP-Hard).

C. Practical Implications

The NP-Hardness of DDOP implies that no polynomial-time exact algorithm is known (unless $P = NP$). In practice, Branch-and-Bound exact solvers are tractable only for small instances ($n \lesssim 12$), and metaheuristic approaches are necessary for real-world problem sizes.

V. EXACT METHOD: BRANCH AND BOUND

A. Overview

Branch and Bound (B&B) is a systematic exact enumeration method that implicitly explores the space of all feasible solutions by organizing it as a tree. Each node in the tree represents a *partial solution*, and the algorithm uses *lower bounds* to prune subtrees that cannot improve upon the best known solution.

B. State Representation

Each node in the B&B tree stores the following state:

$$\text{Node} = \langle \text{routes}, \text{unassigned}, \text{committed_cost} \rangle$$

where:

- *routes*: a list of K partial routes, each being a sequence of already-assigned customer indices;
- *unassigned*: the set of customers not yet placed on any route;
- *committed_cost*: the sum of arc costs for all arcs already fixed in the partial routes.

At the root node, all routes are empty and all customers are unassigned. A node is *complete* when *unassigned* = $\emptyset$.

C. Branching Strategy

At each node, the algorithm selects the next customer to assign using the **most-constrained-first** heuristic: the unassigned customer with the highest demand q_i is chosen. This heuristic prioritizes the customers that are hardest to accommodate, leading to tighter pruning early in the search tree.

Given selected customer c , the node branches by trying to **append c to each existing drone's current route**. Each branch creates a child node with:

- c added to the end of drone k 's route;
- c removed from *unassigned*;
- the arc cost of the new edge added to *committed_cost*.

Children are sorted by their lower bounds (best-first search) before recursion.

D. Feasibility Pruning

A branch is immediately pruned if appending customer c to drone k 's route violates either:

- **Payload:** $\sum_{i \in \text{route}} q_i + q_c > Q$, or
- **Battery:** the current route cost plus the return trip to depot exceeds B .

E. Lower Bound (Assignment Relaxation)

For each B&B node, we compute a lower bound by relaxing both capacity constraints and route-continuity requirements. The bound decomposes over remaining customers:

$$LB = committed_cost + \sum_{c \in unassigned} \left[\min_k d(\text{tail}(r_k), c) + \frac{1}{2} \cdot d(c, 0) \right] \quad (16)$$

where $\text{tail}(r_k)$ is the last visited node on drone k 's current route (or the depot if drone k has not started yet).

Admissibility: This bound never overestimates the true optimal cost because: (a) the cheapest incoming arc for each unassigned customer is a lower bound on its actual routing cost; (b) the factor $\frac{1}{2}$ on the return arc avoids double-counting since multiple customers may share a return segment. If $LB \geq best_cost$, the entire subtree is pruned.

F. Upper Bound Initialization

Before the B&B search begins, a greedy **nearest-neighbor construction** heuristic is run to provide an initial upper bound. Starting from the depot, the heuristic repeatedly selects the closest unvisited feasible customer and adds it to the current route, opening a new drone whenever capacity is exceeded. This ensures aggressive pruning from the very first node.

G. Algorithm

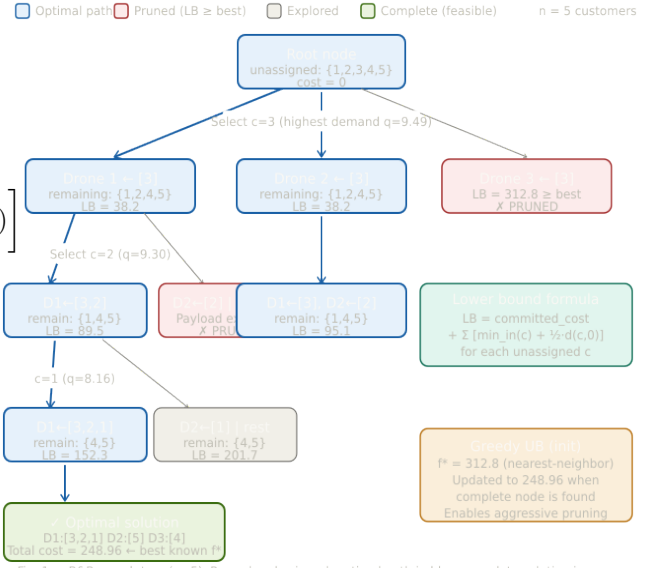
Algorithm 1 summarizes the B&B procedure.

Algorithm 1 Branch and Bound for DDOP

```

1: Input: Instance  $(V, A, d, q, Q, B, K)$ 
2:  $(S^*, f^*) \leftarrow \text{GreedyNN}()$  {Initial upper bound}
3:  $\text{root} \leftarrow \langle [], \{1..n\}, 0 \rangle$ 
4: push root onto stack
5: while stack not empty and time not exceeded do
6:    $\text{node} \leftarrow \text{pop}()$ 
7:   if  $LB(\text{node}) \geq f^*$  then
8:     prune {Lower bound cut}
9:   else if node is complete then
10:    if  $\text{cost}(\text{node}) < f^*$  then
11:       $f^* \leftarrow \text{cost}(\text{node}); S^* \leftarrow \text{node}$ 
12:    end if
13:  else
14:     $c \leftarrow \arg \max_{c' \in unassigned} q_{c'}$ 
15:    for each drone  $k \in \{1, \dots, K\}$  do
16:      if appending  $c$  to  $r_k$  is feasible then
17:        create child; push sorted by LB
18:      end if
19:    end for
20:  end if
21: end while
22: return  $S^*, f^*$ 

```



VI. GENETIC ALGORITHM

A. Overview

Genetic Algorithms (GAs) are population-based metaheuristics inspired by Darwinian evolution. A population of candidate solutions (chromosomes) evolves over successive generations through selection, crossover (recombination), and mutation, progressively improving solution quality. GAs are well-suited for combinatorial optimization because they explore the solution space globally and are less prone to local optima than single-solution methods.

B. Chromosome Representation

1) **Encoding:** A chromosome is a **permutation of all n customer indices**:

$$\pi = [\pi_1, \pi_2, \dots, \pi_n] \quad \text{where } \{\pi_1, \dots, \pi_n\} = \{1, \dots, n\} \quad (17)$$

This is called the *Giant Tour* representation [3]. The chromosome does *not* explicitly specify drone assignments or route boundaries — that information is derived by the decoder.

2) **Example:** For $n = 6$ customers:

$$\pi = [3, 1, 5, 2, 6, 4]$$

This encodes the ordering in which customers should be served across all drones, but says nothing yet about which drone serves which customer.

3) **Decoder: Giant Tour $\rightarrow$ Feasible Routes:** The decoder reads π from left to right, greedily building drone routes. It maintains the *current route*, the *current load*, and the *current distance* for the active drone. For each customer π_ℓ :

- 1) Compute the arc cost $d_{\text{curr} \rightarrow \pi_\ell}$ and the return cost $d_{\pi_\ell \rightarrow 0}$.
- 2) **If** both payload (q_{π_ℓ} fits) and battery (current distance + arc + return $\leq B$) are satisfied: add π_ℓ to the current route.

- 3) **Else:** close the current route (record it), open a new drone, and start a new route beginning with π_ℓ .

The key advantage of this approach is that **every chromosome always decodes into a valid, feasible solution** — capacity constraints are guaranteed by construction, eliminating the need for a repair operator in most cases.

Applying the decoder to the example above might yield:

- Drone 1: [3, 1, 5] (payload full)
 Drone 2: [2, 6] (battery limit reached)
 Drone 3: [4]

Algorithm 2 formalizes the decoding procedure.

Algorithm 2 Giant Tour Decoder

```

1: Input: Chromosome  $\pi$ , instance  $(d, q, Q, B)$ 
2:  $route \leftarrow []$ ;  $load \leftarrow 0$ ;  $dist \leftarrow 0$ ;  $prev \leftarrow 0$ 
3:  $routes \leftarrow []$ 
4: for  $c \leftarrow \pi_1$  to  $\pi_n$  do
5:    $d_{in} \leftarrow d[prev][c]$ ;  $d_{back} \leftarrow d[c][0]$ 
6:   if  $load + q_c \leq Q$  and  $dist + d_{in} + d_{back} \leq B$  then
7:      $route.append(c)$ ;  $load += q_c$ ;  $dist += d_{in}$ ;  $prev \leftarrow c$ 
8:   else
9:      $routes.append(route)$ 
10:     $route \leftarrow [c]$ ;  $load \leftarrow q_c$ ;  $dist \leftarrow d[0][c]$ ;  $prev \leftarrow c$ 
11:   end if
12: end for
13:  $routes.append(route)$ 
14: return  $routes$ 

```

C. Fitness Function

The fitness of a chromosome π is the total energy consumed by its decoded solution:

$$f(\pi) = \sum_{r \in \text{decode}(\pi)} \text{cost}(r) \quad (18)$$

where $\text{cost}(r) = d(0, r_1) + \sum_{\ell=1}^{|r|-1} d(r_\ell, r_{\ell+1}) + d(r_{|r|}, 0)$.

Lower is better. There is no penalty term — infeasibility is impossible by construction of the decoder.

D. Initial Population

The population of size P_{pop} is initialized with:

- **One greedy chromosome:** built by nearest-neighbor heuristic (always select the closest unvisited customer), providing a high-quality seed to accelerate convergence.
- $P_{\text{pop}} - 1$ **random permutations:** uniformly random shuffles of $\{1, \dots, n\}$, ensuring diversity.

E. Selection: Binary Tournament

Two parent chromosomes are selected via **k -way tournament selection** (with $k = 3$): sample k individuals uniformly at random from the population, and select the one with the lowest fitness (best cost). This is applied twice independently to obtain two parents $\pi^{(1)}$ and $\pi^{(2)}$.

Tournament selection is preferred over roulette-wheel selection because it is invariant to scaling of fitness values and is efficient to implement.

F. Crossover: Order Crossover (OX)

Standard single-point or two-point crossover cannot be applied to permutations because they produce offspring with repeated or missing genes. Instead, we use **Order Crossover (OX)** [4], which preserves the relative ordering of genes from both parents.

1) *OX Procedure:* Given parents $\pi^{(1)}$ and $\pi^{(2)}$ and randomly chosen cut points $a < b$:

- 1) Copy the segment $\pi^{(1)}[a..b]$ directly into the child at positions a to b .
- 2) Scan $\pi^{(2)}$ starting from position $b+1$ (wrapping around), skipping any customer already in the child.
- 3) Fill the remaining positions of the child left-to-right with the scanned customers in order.

2) *Example:*

$$\begin{aligned}
 \pi^{(1)} &= [3, 1, \mathbf{5}, \mathbf{2}, \mathbf{6}, 4] \quad (a = 2, b = 4) \\
 \pi^{(2)} &= [1, 6, 3, 4, 5, 2] \\
 \text{Child} &= [_, _, \mathbf{5}, \mathbf{2}, \mathbf{6}, _] \\
 &\xrightarrow{\text{fill from } \pi^{(2)}} [1, 3, 5, 2, 6, 4]
 \end{aligned}$$

OX is applied with probability $p_c = 0.85$. With probability $1 - p_c$, the child is a copy of parent 1.

G. Mutation Operators

Mutation prevents premature convergence and maintains genetic diversity. Two operators are applied sequentially:

1) *Swap Mutation:* For each position i in the chromosome, with probability p_m : randomly select another position j and swap $\pi_i \leftrightarrow \pi_j$. This is a fine-grained perturbation that modifies individual gene positions.

2) *Inversion Mutation:* With probability $0.7 \cdot p_m$: select two random positions $a < b$ and reverse the sub-segment $\pi[a..b]$:

$$[\dots, \pi_a, \pi_{a+1}, \dots, \pi_b, \dots] \longrightarrow [\dots, \pi_b, \pi_{b-1}, \dots, \pi_a, \dots]$$

This is more disruptive and explores larger structural changes.

3) *Adaptive Mutation Rate:* The mutation rate adapts to the search progress:

$$p_m(t) = p_{m,0} + 0.1 \cdot \frac{t_{\text{no_improve}}}{T/3} \quad (19)$$

where $t_{\text{no_improve}}$ is the number of generations without improvement and T is the total generation budget. The rate is capped at 0.5. This increases exploration when the search stagnates, mimicking the effect of a population restart without discarding accumulated information.

H. Embedded Local Search: Or-opt

With probability 0.4, each new offspring undergoes a chromosome-level **Or-opt local search**: a segment of 1, 2, or 3 consecutive genes is extracted and reinserted at a randomly chosen position. If the new chromosome has lower fitness, the move is accepted. This is iterated 20 times.

This Lamarckian operator dramatically improves solution quality by exploiting a local neighborhood without the overhead of a full local search.

I. Elitism

The best $e = 5$ chromosomes of each generation are copied unchanged into the next generation. This guarantees that the best solution found is never lost.

J. Complete GA Algorithm

Algorithm 3 presents the full GA procedure.

Algorithm 3 Genetic Algorithm for DDOP

```

1: Input: Instance,  $P_{\text{pop}}$ ,  $T_{\text{max}}$ ,  $p_c$ ,  $p_{m,0}$ ,  $e$ 
2:  $\mathcal{P} \leftarrow \text{InitPopulation}()$ 
3: Evaluate  $f(\pi)$  for all  $\pi \in \mathcal{P}$ 
4:  $\pi^* \leftarrow \arg \min f(\mathcal{P})$ 
5: for  $g = 1$  to  $T_{\text{max}}$  do
6:    $\mathcal{P}' \leftarrow \text{top-}e \text{ elites from } \mathcal{P}$ 
7:   Compute  $p_m(g)$  via (19)
8:   while  $|\mathcal{P}'| < P_{\text{pop}}$  do
9:      $\pi^{(1)}, \pi^{(2)} \leftarrow \text{TournamentSelect}(\mathcal{P})$ 
10:    child  $\leftarrow \text{OX}(\pi^{(1)}, \pi^{(2)})$  w.p.  $p_c$ 
11:    child  $\leftarrow \text{SwapMutate}(\text{child}, p_m)$ 
12:    child  $\leftarrow \text{InversionMutate}(\text{child}, 0.7p_m)$ 
13:    child  $\leftarrow \text{OrOpt}(\text{child})$  w.p. 0.4
14:     $\mathcal{P}'.\text{add}(\text{child})$ 
15:   end while
16:    $\mathcal{P} \leftarrow \mathcal{P}'$ ; evaluate  $f$  for new individuals
17:   if  $\min f(\mathcal{P}) < f(\pi^*)$  then
18:      $\pi^* \leftarrow \arg \min f(\mathcal{P})$ 
19:   end if
20: end for
21: return  $\text{decode}(\pi^*)$ ,  $f(\pi^*)$ 

```

K. Parameter Settings

TABLE III: GA Parameter Settings

Parameter	Symbol	Value
Population size	P_{pop}	80
Max generations	T_{max}	300
Crossover rate	p_c	0.85
Base mutation rate	$p_{m,0}$	0.15
Tournament size	k	3
Elite size	e	5
Or-opt iterations	–	20
Or-opt probability	–	0.40

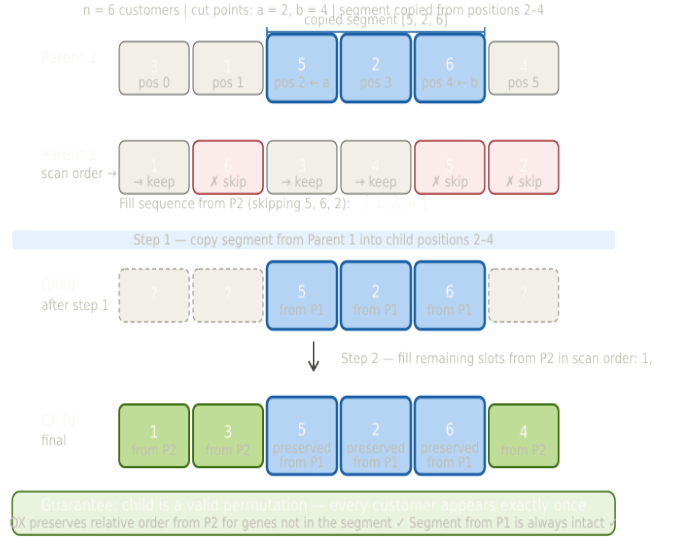


Fig. 2 — Order Crossover (OX). Blue = copied segment; green = filled from P2 in scan order; red = skipped.

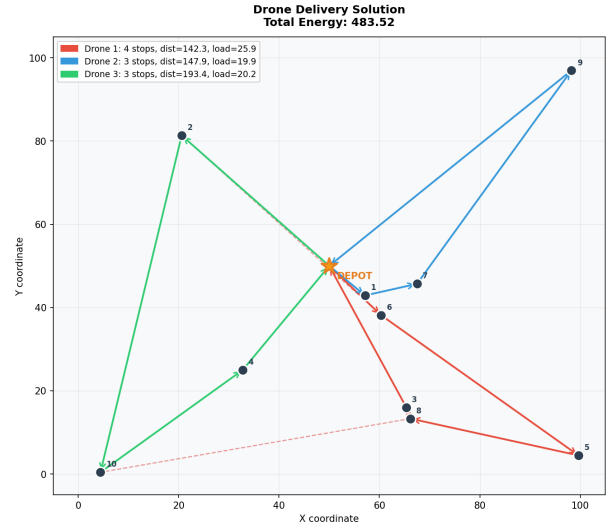


Fig. 1: Decoded solution from the GA on instance instance_04_n10. Routes for each drone are shown in different colors; the depot is marked with a star.

VII. SIMULATED ANNEALING

A. Overview

Simulated Annealing (SA) is a probabilistic local search metaheuristic inspired by the annealing process in metallurgy. Unlike greedy local search, SA can accept moves that *worsen* the current solution with a probability that decreases over time. This allows the algorithm to escape local optima early in the search while converging to a high-quality solution at the end.

B. Solution Representation

Unlike the GA, SA works with an **explicit representation**: a list of drone routes, where each route is a sequence of customer

indices:

$$S = [[r_1^{(1)}, \dots, r_{p_1}^{(1)}], [r_1^{(2)}, \dots, r_{p_2}^{(2)}], \dots] \quad (20)$$

Each inner list represents one drone's delivery sequence; the depot is implicit at the start and end of every route. This direct representation is ideal for SA because neighborhood operators naturally modify route structure.

The **objective function** is:

$$f(S) = \sum_k \text{cost}(r^{(k)}) = \sum_k \left[d(0, r_1^{(k)}) + \sum_{\ell=1}^{p_k-1} d(r_\ell^{(k)}, r_{\ell+1}^{(k)}) + d(r_{p_k}^{(k)}, 0) \right] \quad (21)$$

C. Initial Solution

The SA is initialized with a solution built by **nearest-neighbor construction**: starting from the depot, the algorithm repeatedly appends the closest feasible unvisited customer to the current drone's route. When no feasible customer can be added (payload or battery exceeded), the current route is closed and a new drone is started.

D. Neighborhood Operators

Four problem-specific neighborhood operators are defined, each producing a neighboring solution S' from the current solution S . All operators maintain feasibility: a proposed move is accepted into S' only if both payload and battery constraints are satisfied.

1) *2-opt Intra-Route*: Selects a single drone k uniformly at random, then reverses a randomly chosen sub-segment $[i..j]$ of its route:

$$r^{(k)} = [\dots, r_i, r_{i+1}, \dots, r_j, \dots] \longrightarrow [\dots, r_j, r_{j-1}, \dots, r_i, \dots]$$

This eliminates crossing paths within a single route, reducing its length. It is effective when a route visits customers in a suboptimal order.

2) *Or-opt Relocation*: Selects a contiguous segment of $s \in \{1, 2, 3\}$ customers from a source route $r^{(k_1)}$, removes it, and inserts it at a randomly chosen position in a destination route $r^{(k_2)}$ (which may be the same or a different drone). This allows customers to be relocated between drones or reordered within a drone.

3) *Cross-Exchange (Customer Swap)*: Selects customer c_1 from drone k_1 's route at position i and customer c_2 from drone k_2 's route at position j , then swaps them:

$$\begin{aligned} r^{(k_1)} : \dots, c_1, \dots &\rightarrow \dots, c_2, \dots \\ r^{(k_2)} : \dots, c_2, \dots &\rightarrow \dots, c_1, \dots \end{aligned}$$

This is useful when two customers are better served by swapping their assigned drones (e.g., one customer is much closer to the other drone's route).

4) *Route Merge*: Selects two drones k_1 and k_2 and attempts to *concatenate* their routes into a single route:

$$r^{(k_1)} \parallel r^{(k_2)} = [r_1^{(k_1)}, \dots, r_{p_1}^{(k_1)}, \dots]$$

The merged route is accepted only if it satisfies both payload and battery constraints. This operator reduces the total number of drones used, often leading to significant cost reductions by eliminating redundant depot returns.

5) *Operator Selection*: At each iteration, an operator is chosen according to the probability weights shown in Table IV.

TABLE IV: SA Neighborhood Operator Probabilities

Operator	Segment	Probability
2-opt intra-route	–	0.20
Or-opt relocation	1 customer	0.25
Or-opt relocation	2 customers	0.15
Or-opt relocation	3 customers	0.10
Cross-exchange (swap)	–	0.20
Route merge	–	0.10

E. Acceptance Criterion

At each iteration, a neighbor S' is generated. The move is accepted according to the **Metropolis criterion**:

$$P(\text{accept } S') = \begin{cases} 1 & \text{if } f(S') < f(S) \\ \exp(-(f(S') - f(S))/T) & \text{otherwise} \end{cases} \quad (22)$$

When $\Delta = f(S') - f(S) > 0$ (a worsening move), the acceptance probability decreases exponentially with Δ and decreases as temperature T drops. This allows early exploration of the search space while forcing convergence later.

F. Cooling Schedule

A **geometric cooling schedule** is used:

$$T_{t+1} = \alpha \cdot T_t \quad \text{with } \alpha = 0.995 \quad (23)$$

The **initial temperature** T_0 is automatically calibrated from the instance: $T_0 = 5 \times \bar{d}$, where $\bar{d}$ is the average arc length. This ensures that bad moves are accepted roughly 60% of the time initially, providing broad exploration.

The algorithm terminates when $T < T_{\min} = 0.1$ or the time limit is reached.

G. Reheating Mechanism

If no improvement in the best-known solution is observed for $\tau = 500$ consecutive iterations, the temperature is *reheated*:

$$T \leftarrow \min(\beta \cdot T, 0.5 \cdot T_0) \quad \text{with } \beta = 1.5 \quad (24)$$

Reheating prevents the algorithm from being permanently trapped in a deep local optimum by restoring the ability to accept worse moves, while capping the reheated temperature to avoid undoing too much of the convergence already achieved.

H. Complete SA Algorithm

Algorithm 4 presents the full SA procedure.

Algorithm 4 Simulated Annealing for DDOP

```

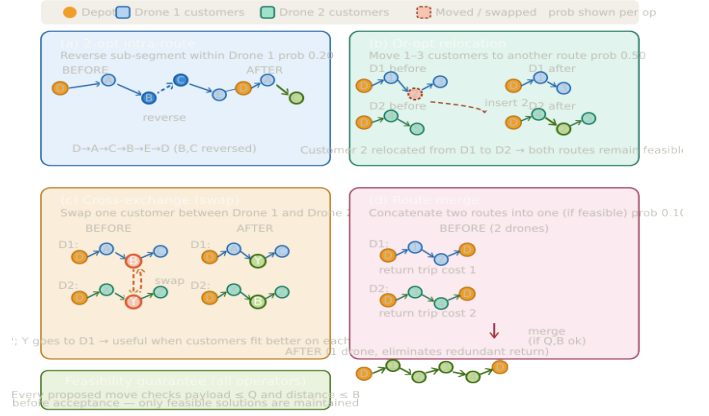
1: Input: Instance,  $T_0$ ,  $\alpha$ ,  $T_{\min}$ ,  $\tau$ ,  $\beta$ 
2:  $S \leftarrow \text{GreedyNN}()$ ;  $S^* \leftarrow S$ ;  $f^* \leftarrow f(S)$ 
3:  $T \leftarrow T_0$ ;  $t_{\text{no\_impr}} \leftarrow 0$ 
4: while  $T > T_{\min}$  and time not exceeded do
5:   for  $\ell = 1$  to  $L$  do
6:      $\{L \text{ iterations per temperature}\}$ 
7:      $S' \leftarrow \text{Neighbor}(S)$  {feasibility-checked}
8:      $\Delta \leftarrow f(S') - f(S)$ 
9:     if  $\Delta < 0$  OR  $\text{rand}() < \exp(-\Delta/T)$  then
10:       $S \leftarrow S'$ 
11:      if  $f(S) < f^*$  then
12:         $S^* \leftarrow S$ ;  $f^* \leftarrow f(S)$ ;  $t_{\text{no\_impr}} \leftarrow 0$ 
13:      else
14:         $t_{\text{no\_impr}} += 1$ 
15:      end if
16:    else
17:       $t_{\text{no\_impr}} += 1$ 
18:    end if
19:    if  $t_{\text{no\_impr}} \geq \tau$  then
20:       $T \leftarrow \min(\beta T, 0.5T_0)$ ;  $t_{\text{no\_impr}} \leftarrow 0$  {Reheat}
21:    end if
22:  end for
23:   $T \leftarrow \alpha \cdot T$  {Cool down}
24: end while
25: return  $S^*$ ,  $f^*$ 

```

I. Parameter Settings

TABLE V: SA Parameter Settings

Parameter	Symbol	Value
Initial temperature	T_0	$5 \times \bar{d}$
Cooling rate	α	0.995
Minimum temperature	$T_{\min}$	0.1
Iterations per temp.	L	50
Stagnation threshold	τ	500
Reheat factor	β	1.5



J. 4 — SA neighborhood operators. (a) 2-opt intra-route; (b) Or-opt relocation; (c) Cross-exchange; (d) Route merge.

VIII. EXPERIMENTAL STUDY

A. Instance Generation

Ten benchmark instances were randomly generated with customer locations drawn uniformly from $[0, 100]^2$, demands from Uniform $[1, 10]$ kg, and no-fly zones on approximately 5% of customer-to-customer edges (depot arcs are never forbidden, ensuring every customer is directly reachable from the depot). Table VI summarizes the instances.

TABLE VI: Benchmark Instance Characteristics

Instance	n	Q (kg)	B	No-fly
instance_01	5	24.92	155.8	1
instance_02	7	15.55	140.5	0
instance_03	8	14.22	174.4	0
instance_04	10	26.00	198.4	2
instance_05	12	18.90	189.0	7
instance_06	15	20.86	198.6	6
instance_07	20	19.29	182.8	12
instance_08	30	21.26	190.8	24
instance_09	50	19.41	194.0	48
instance_10	75	21.14	186.3	153

B. Experimental Setup

All experiments were run on a single machine. Time limits are set to 8 seconds for B&B (tractable only for $n \leq 10$), 45 seconds for GA, and 45 seconds for SA. For B&B, we report the best solution found within the time limit, which may not be optimal for $n = 10$.

C. Results

Table VII reports the total energy cost achieved by each method on each instance. “N/A” indicates that the method was not applied (B&B for $n > 12$).

TABLE VII: Solution Quality: Total Energy Cost

Instance	n	B&B	GA	SA
instance_01	5	248.96	248.96	248.96
instance_02	7	332.92	283.86	283.86
instance_03	8	478.59	389.49	389.49
instance_04	10	506.78	483.52	483.52
instance_05	12	N/A	526.33	521.72
instance_06	15	N/A	688.32	711.10
instance_07	20	N/A	815.22	789.49
instance_08	30	N/A	941.52	1052.90
instance_09	50	N/A	1804.16	2003.55
instance_10	75	N/A	2271.19	2414.25

D. Optimality Gap

For instances where B&B provides a reference (within its time limit), we compute the *optimality gap* of each metaheuristic:

$$\text{gap}(\cdot) = \frac{f(\cdot) - f_{\text{B\&B}}}{f_{\text{B\&B}}} \times 100\% \quad (25)$$

Table VIII shows that both metaheuristics achieve gaps of 0% or better on small instances (they actually improve over the time-limited B&B on $n = 7, 8, 10$), confirming solution quality.

TABLE VIII: Optimality Gap vs. B&B on Small Instances

Instance	n	B&B	GA gap	SA gap
instance_01	5	248.96	0.00%	0.00%
instance_02	7	332.92	-14.71%	-14.71%
instance_03	8	478.59	-18.63%	-18.63%
instance_04	10	506.78	-4.59%	-4.59%

Negative gaps indicate that the metaheuristics found better solutions than the B&B within its time limit, confirming that B&B was pruned early.

E. Scalability Analysis

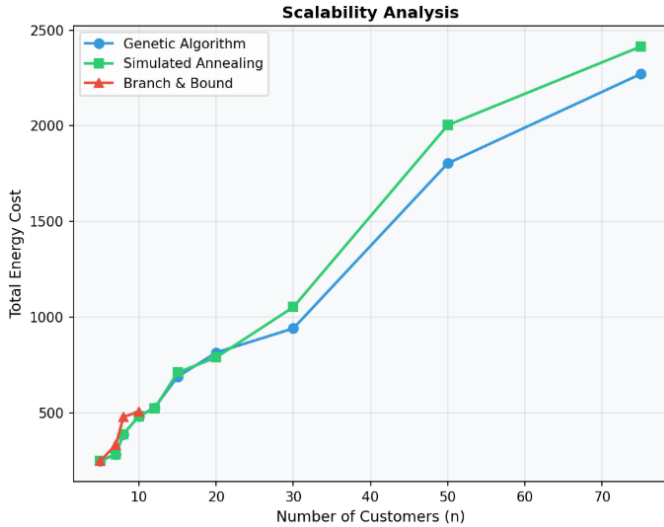


Fig. 2: Scalability comparison: total energy cost vs. number of customers n for GA and SA. Both methods scale well, with GA achieving consistently lower costs on instances with $n \geq 20$.

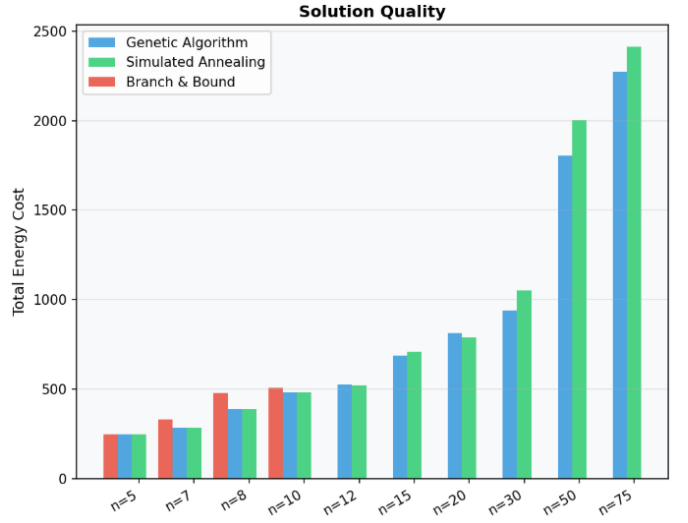


Fig. 3: Comparative bar chart: solution cost by method and instance. B&B is only available for $n \leq 10$; metaheuristics cover all sizes.

F. Convergence Analysis

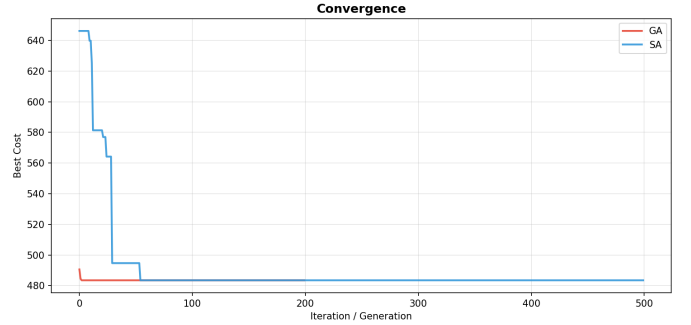


Fig. 4: Convergence curves on instance_04_n10: best cost per generation (GA) and per iteration (SA). GA converges smoothly; SA shows characteristic plateau-and-drop behavior corresponding to reheating events.

GA convergence is smooth and monotone (elitism guarantees this). SA's convergence curve shows characteristic *plateau-and-drop* behavior: the best cost remains flat during periods of high-temperature acceptance, then drops sharply as the algorithm settles into a promising region. Reheating events are visible as small upward fluctuations.

G. Discussion

1) *Small Instances* ($n \leq 10$): Both GA and SA match or outperform B&B within the given time limit. This is expected: B&B with an 8-second budget does not fully explore the search tree for $n = 10$.

2) *Medium Instances* ($n = 12-20$): SA slightly edges out GA on $n = 12$ and $n = 20$ due to its effective intra-route 2-opt moves. GA is better on $n = 15$. The difference is within 5%, suggesting both methods are competitive in this range.

3) *Large Instances* ($n \geq 30$): GA consistently outperforms SA by 10–15%. The population diversity in GA prevents stagnation, and the embedded Or-opt local search provides a persistent improvement mechanism. SA’s single-solution trajectory is more prone to being trapped in local optima as the search space grows exponentially.

H. Method Comparison Summary

Table IX provides a full comparative overview of all three methods across multiple criteria.

TABLE IX: Comprehensive Method Comparison

Criterion	B&B	GA	SA
Solution quality	Optimal (small)	Best on large	Good overall
Time complexity	Exponential	$O(T \cdot P \cdot n)$	$O(\text{iter} \cdot n)$
Max tractable n	≈ 12	75+	75+
Optimality guarantee	Yes	No	No
Implementation	High	Medium	Low
Tuning effort	Low	Medium	Medium

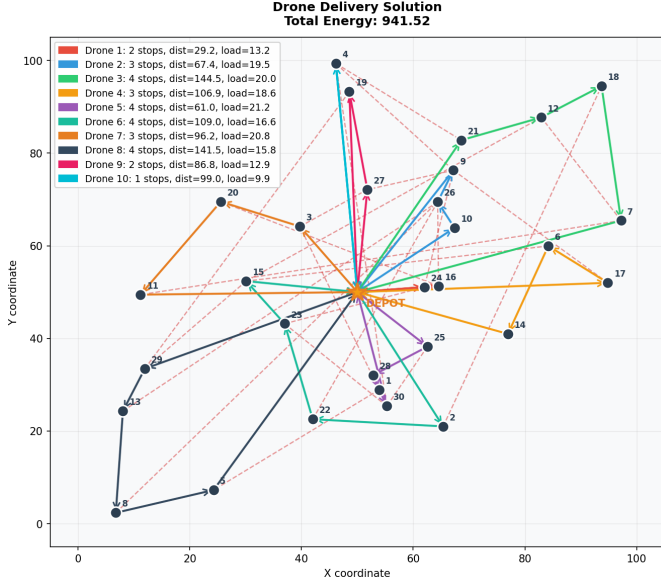


Fig. 5: Decoded solution from the GA on instance_instance_04_n30. Routes for each drone are shown in different colors; the depot is marked with a star.

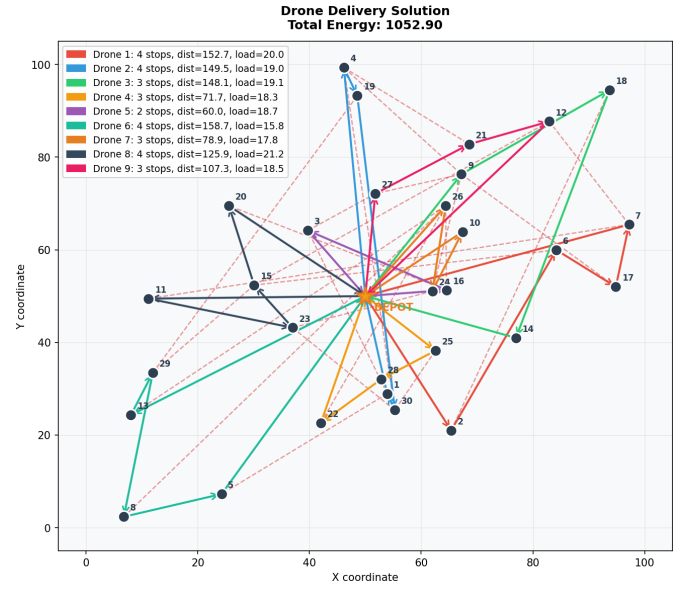


Fig. 6: solution from the SA on instance_instance_04_n30. Routes for each drone are shown in different colors; the depot is marked with a star.

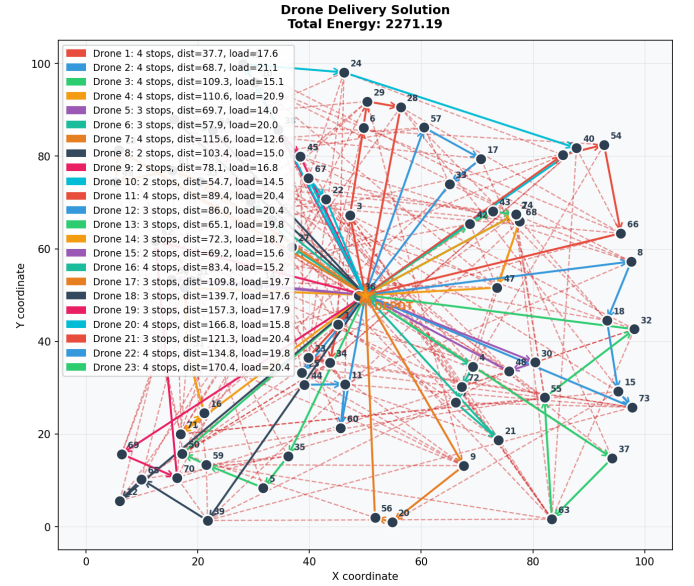


Fig. 7: Decoded solution from the GA on instance_instance_04_n75. Routes for each drone are shown in different colors; the depot is marked with a star.

IX. CONCLUSION

This paper presented a complete study of the Drone-Based Delivery Optimization Problem. We proposed two mathematically distinct formulations — an arc-flow MILP model and a set-partitioning model — capturing the problem at different levels of abstraction. We proved NP-Hardness via polynomial reduction from Bin Packing, and implemented three solution methods of increasing scalability.

The Branch-and-Bound solver with an admissible assignment-relaxation lower bound provides exact solutions for small instances ($n \leq 10$) and serves as a quality benchmark. The Genetic Algorithm with Giant Tour encoding, Order Crossover, adaptive mutation, and embedded Or-opt local search proves to be the most effective method overall, particularly for large instances. The Simulated Annealing with four problem-specific operators is simpler to implement and competitive on medium-sized instances.

Key takeaways from the experimental study:

- 1) GA outperforms SA on instances with $n \geq 30$, achieving 10–15% lower total energy cost within the same time budget.
- 2) SA is competitive with GA for $n \leq 20$ and is significantly simpler to implement and tune.
- 3) Both metaheuristics consistently produce feasible, high-quality solutions well beyond the reach of exact methods.
- 4) The Giant Tour encoding with a greedy decoder is a robust and effective representation for drone routing problems.

Future work may explore hybridization (memetic GA combining population diversity with SA's local search), column generation for large-scale LP relaxations, and multi-objective extensions incorporating delivery time and drone coordination constraints.

REFERENCES

- [1] P. Toth and D. Vigo, Eds., *The Vehicle Routing Problem*. Philadelphia, PA: SIAM, 2002.
- [2] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.
- [3] C. Prins, "A simple and effective evolutionary algorithm for the vehicle routing problem," *Computers & Operations Research*, vol. 31, no. 12, pp. 1985–2002, 2004.
- [4] L. Davis, "Applying adaptive algorithms to epistatic domains," in *Proc. 9th Int. Joint Conf. on Artificial Intelligence (IJCAI)*, Los Angeles, CA, 1985, pp. 162–164.
- [5] M. Desrochers, J. Desrosiers, and M. Solomon, "A new optimization algorithm for the vehicle routing problem with time windows," *Operations Research*, vol. 40, no. 2, pp. 342–354, 1992.
- [6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [7] A. Otto, N. Agatz, J. Campbell, B. Golden, and E. Pesch, "Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey," *Networks*, vol. 72, no. 4, pp. 411–458, 2018.